Blockchain

Hyperledger Fabric

Principales componentes de Fabric

Cliente: Es la aplicación que inicia las transacciones, firmándolas con su identidad y enviándolas a los peers para su ejecución.

Peers: Son los nodos de la red que ejecutan la lógica del negocio (chaincode) y almacenan una copia del ledger.

Orderer: Es el servicio encargado de ordenar las transacciones y agruparlas en bloques que se distribuyen a los peers.

Chaincode: Es el código que define la lógica de negocio y las reglas que determinan cómo cambian los datos en el sistema. Se puede decir que es el equivalente a los Smart Contracts

Ledger: Es la base de datos distribuida que contiene tanto el historial de transacciones como el estado actual del sistema. Cada peer guarda una copia (de los canales en los que participa)

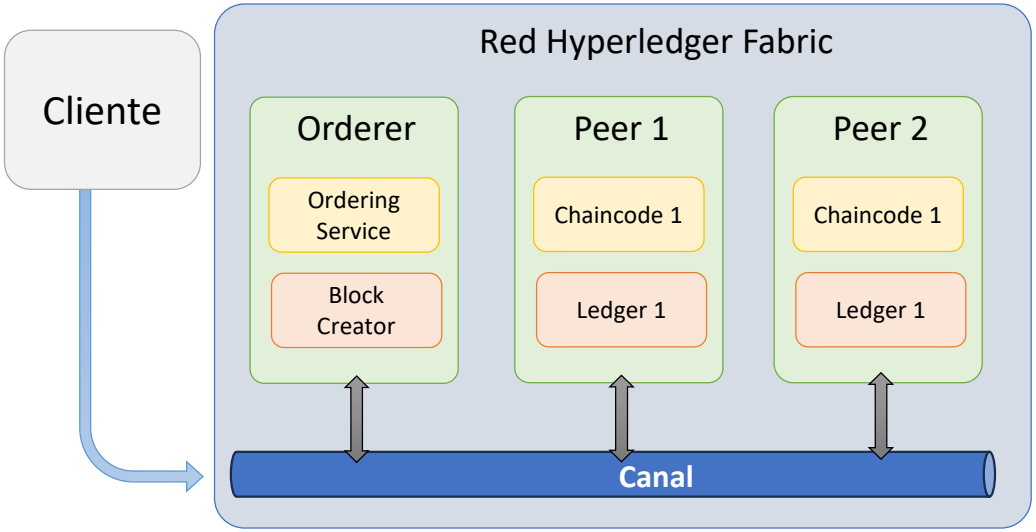
CA (Certificate Authority): Es el componente que emite las identidades digitales (certificados) para todos los participantes de la red.

MSP (Membership Service Provider): Es el sistema que valida las identidades y define qué participantes están autorizados a operar en la red.

Canal: Es un mecanismo que permite crear subredes privadas donde solo ciertos participantes pueden ver y compartir información.

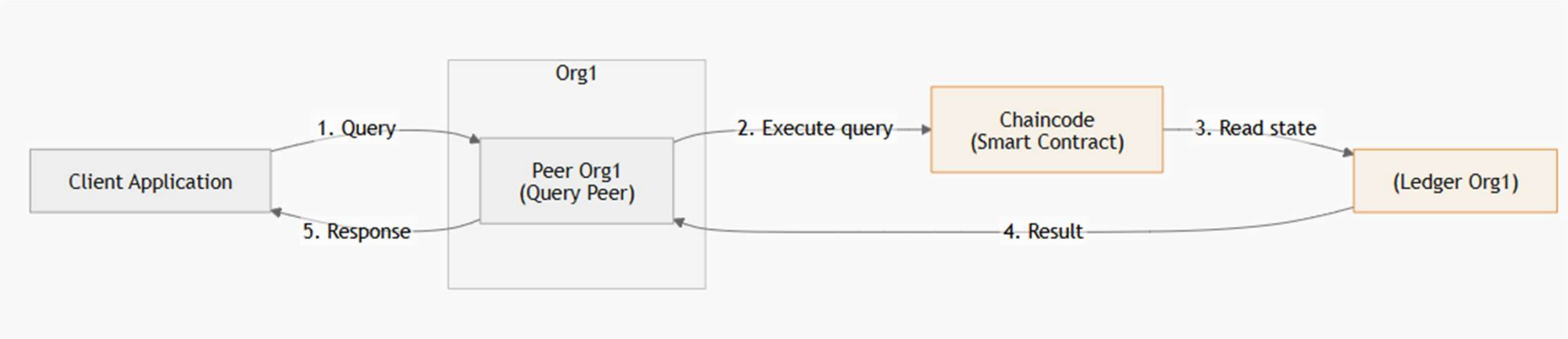
PDC (Private Data Collection): Es un mecanismo que permite que solo un subconjunto de peers acceda a datos sensibles dentro de un mismo canal, manteniendo el resto oculto.

Esquema lógico simplificado de una red Fabric

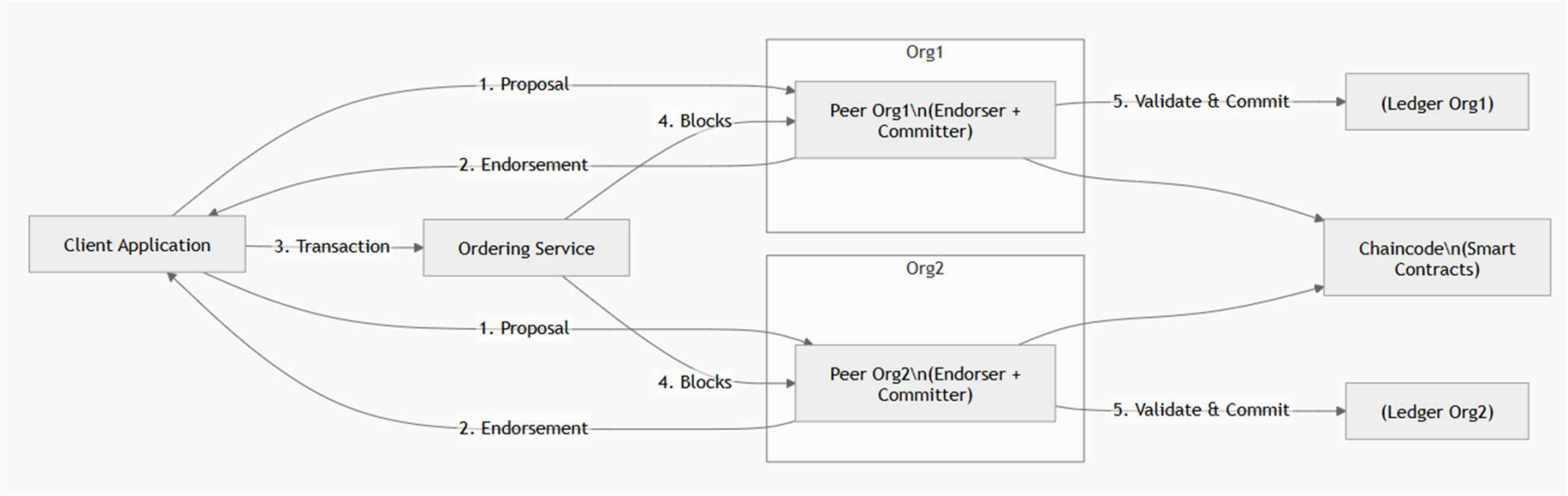


- **Cliente**: aplicación que envía solicitudes a la red.
- **Peer**: nodo que ejecuta chaincode y mantiene el ledger.
- **Chaincode**: lógica de negocio compartida.
- **Ledger**: estado actual e histórico de transacciones.
- **Canal**: espacio privado de comunicación entre miembros.
- **Orderer**: ordena transacciones y crea bloques.
- **Red Fabric**: conjunto de nodos y servicios de la red.

Flujo resumido de una aplicación Fabric (lectura)



Flujo resumido de una aplicación Fabric (escritura)



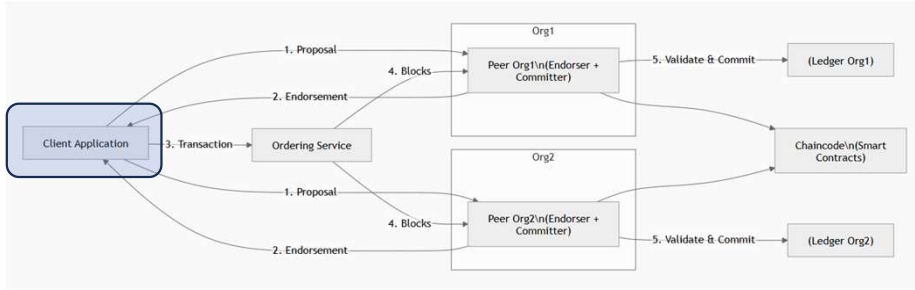
El Cliente

Qué es
- Aplicación que inicia transacciones
- Usa SDK (Node, Java, Go)

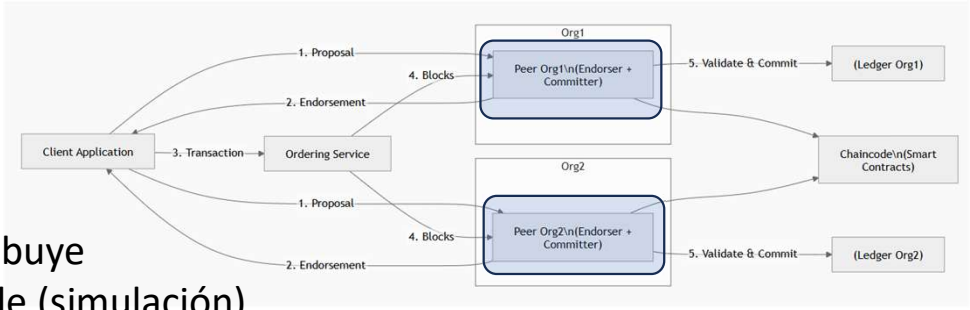
Qué hace
- Construye propuestas
- Firma con su identidad (certificado)
- Envía a peers (endorsers)
- Recoge respuestas y envía al orderer

Importante
- ❌ No ejecuta chaincode
- ❌ No accede al ledger directamente

Idea clave
El cliente orquesta, pero no ejecuta ni valida



Los Peers



Qué son

- Nodos que mantienen el ledger

Roles

- Anchor peer → Peer públicos (conocidos por otras org)
- Leader peer → Recibe los bloques de orderer y los distribuye
- Endorser → Recine trx de los clientes y ejecuta chaincode (simulación)
- Committer → Nodo regular que valida y escribe bloques
- Un mismo peer puede tener varios roles o todos

Qué hacen

- Ejecutan lógica
- Validan transacciones
- Guardan datos

Importante

- Cada peer tiene su propia copia del ledger
- En realidad una copia del ledger por cada canales en los que participa

Idea clave

Los peers son el “motor” del sistema

El Ordering Service

Qué es

- Servicio que ordena transacciones

Qué hace

- Recibe transacciones endosadas
- Las ordena
- Genera bloques y los distribuye
- Gestiona la configuración de canales

Importante

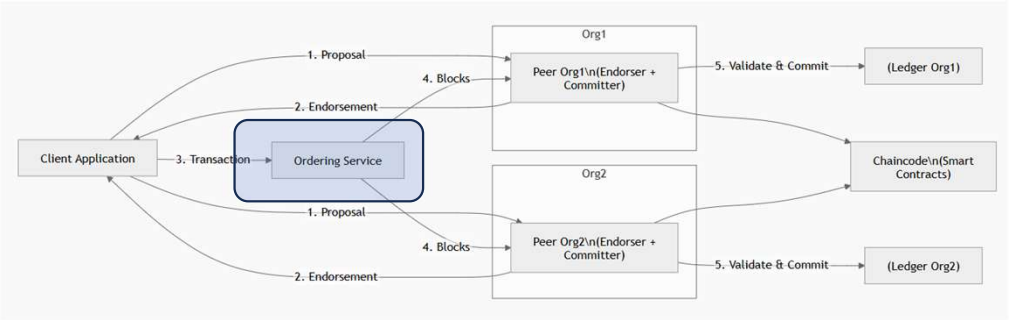
- ❌ No ejecuta chaincode
- ❌ No valida lógica

Gobernanza

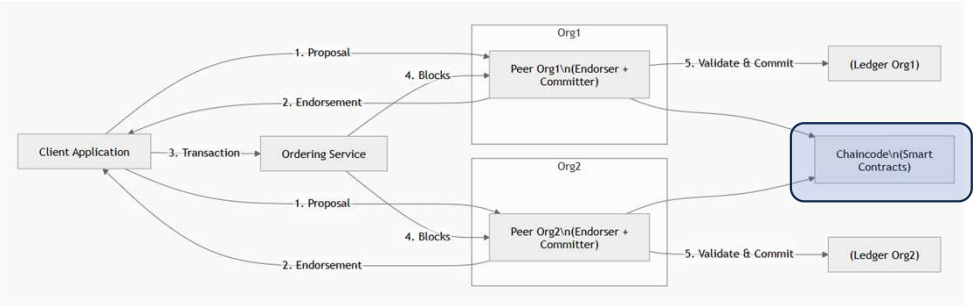
- Operado por varias organizaciones (Raft)

Idea clave

El orderer no decide qué es válido, solo el orden



El Chaincode



Qué es

- Lógica de negocio (smart contracts)

Dónde vive

- En los peers (contenedor / proceso)

Cómo se usa

- Cliente → peer → chaincode
- function + args

Importante

- ❌ No tiene endpoints propios
- ❌ No se ejecuta en cliente

Duda típica

- No hay ABI estándar (como Ethereum) sino una definición de funciones

Idea clave

El chaincode define reglas, pero el peer lo ejecuta

El Ledger

Qué es

- Base de datos distribuida

Componentes

- Blockchain → histórico inmutable
- World State → estado actual

Dónde está

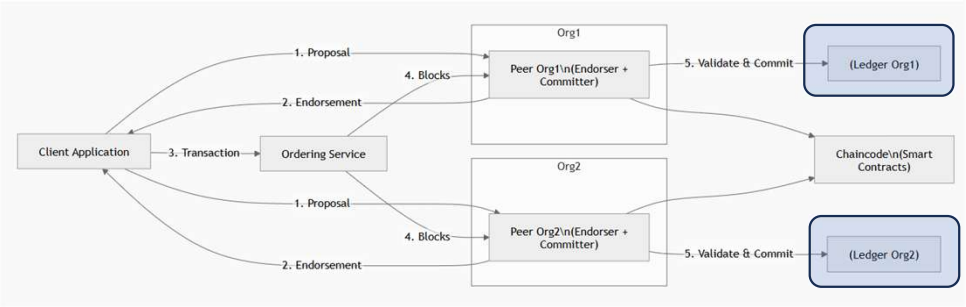
- Dentro de cada peer

Importante

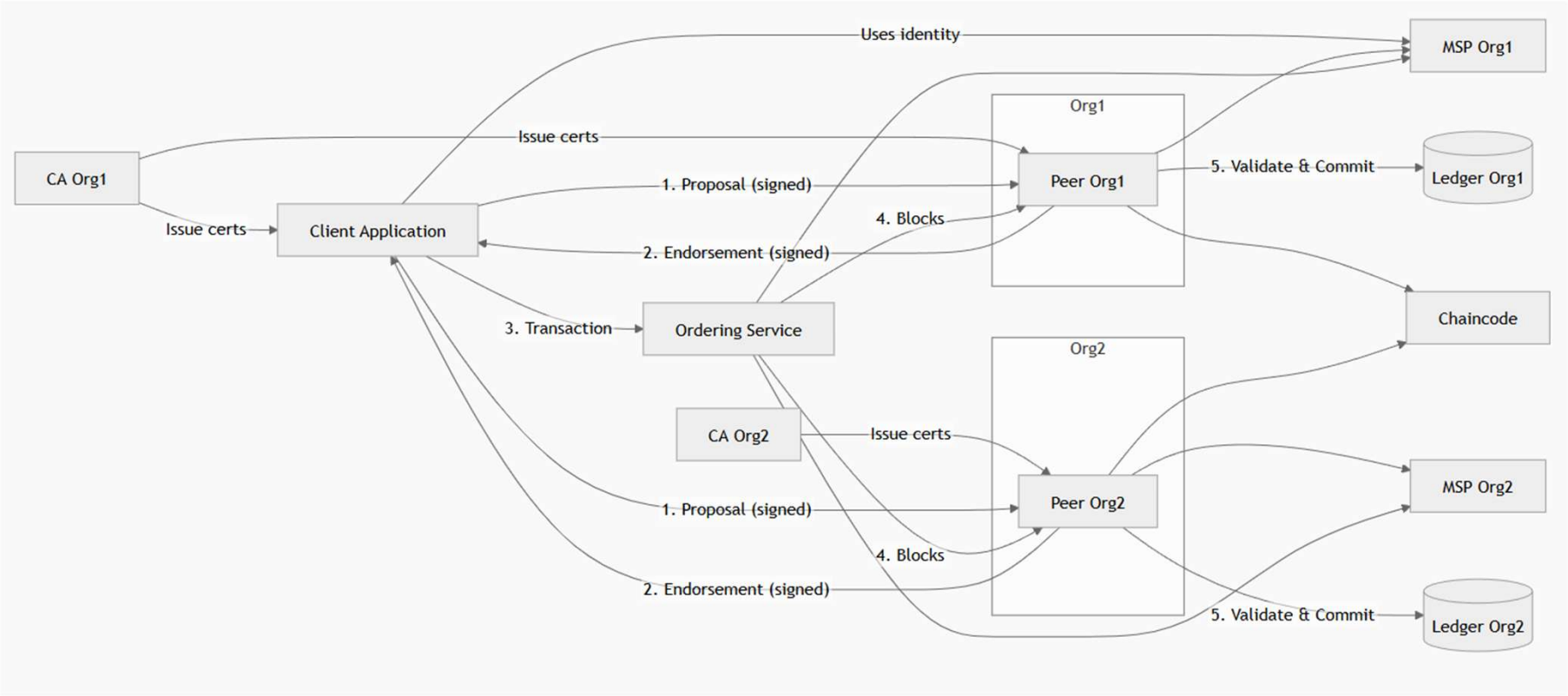
- Un ledger por canal
- No hay ledger global

Idea clave

El ledger es el dato, no el nodo



Flujo detallado de una aplicación Fabric



CA (Certificate Authority)

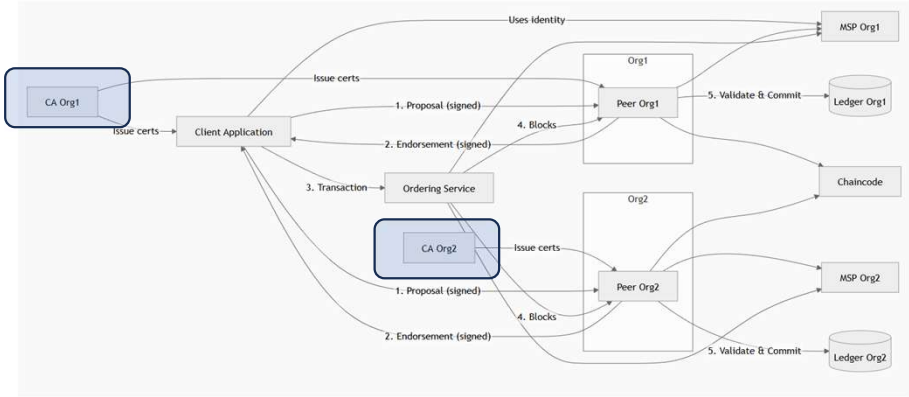
Qué es
- Servicio que emite identidades

Qué hace
- Genera certificados (X.509)
- Identifica usuarios, peers, orderers

Modelo
- Cada organización tiene su CA

Importante
- ❌ No valida transacciones
- ❌ No define confianza global

Idea clave
La CA emite identidades, no decide quién es válido



MSP (Membership Service Provider)

Qué es
- Sistema de validación de identidades

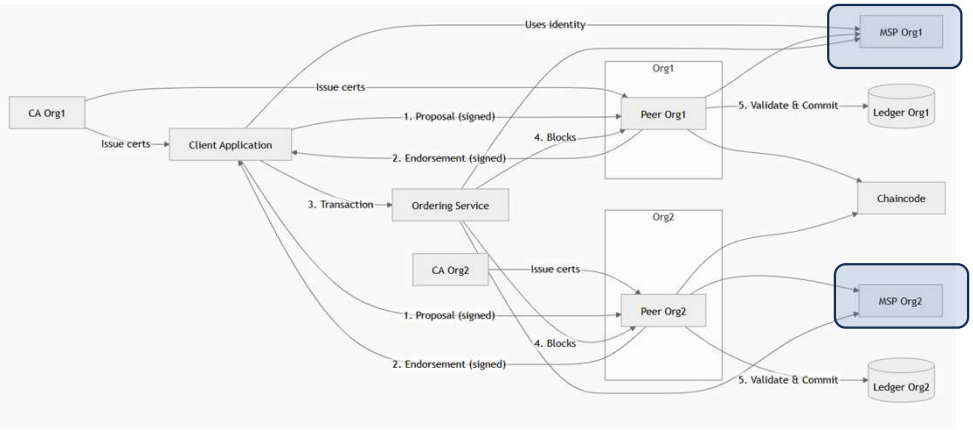
Qué hace
- Define qué CAs son confiables
- Verifica certificados y firmas

Dónde está
- En peers, orderers y clientes

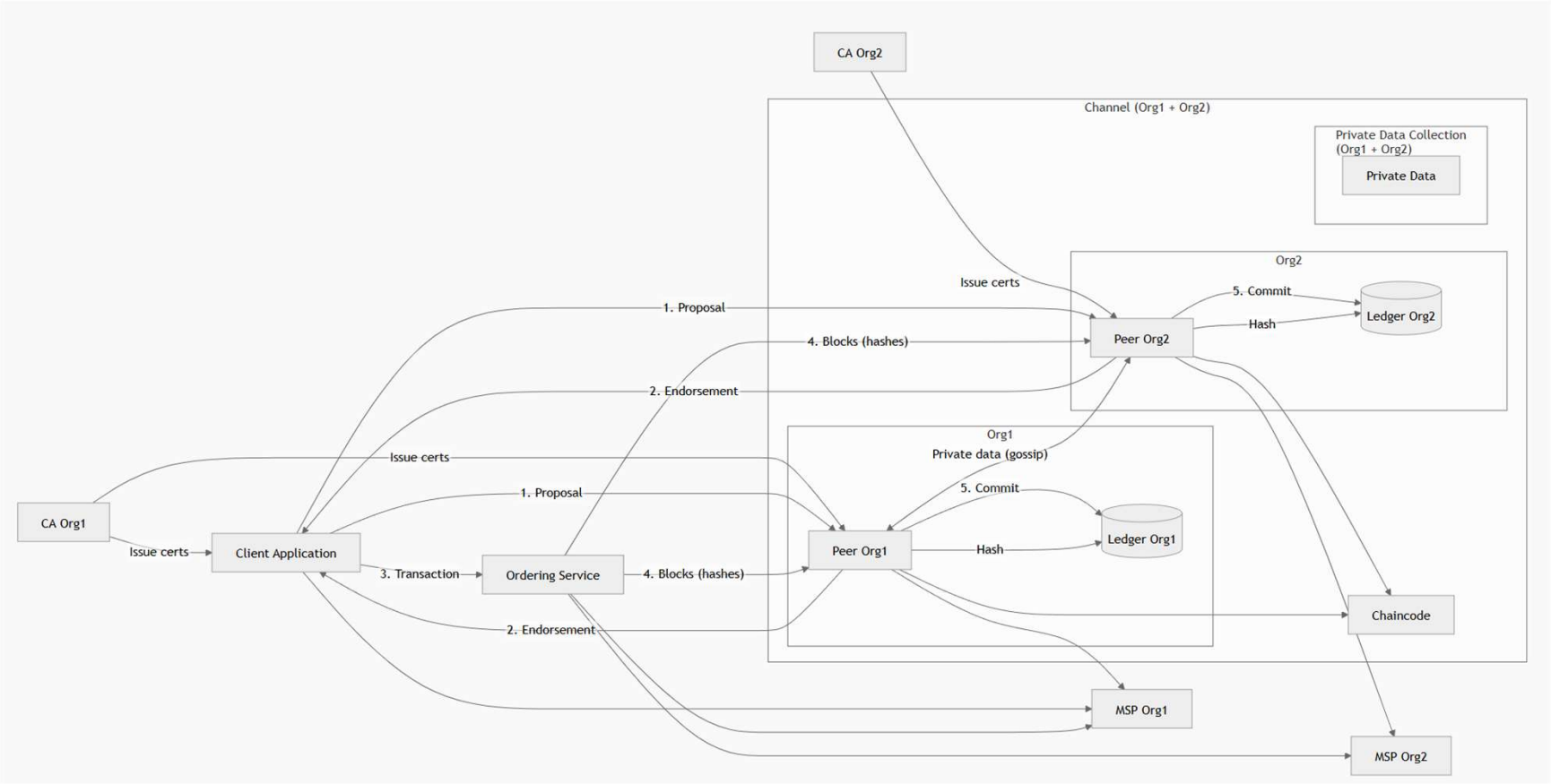
Importante
- ❌ No es un servicio
- ✓ Es configuración

Duda clave
- Validación es local (no llamadas externas)

Idea clave
El MSP define la confianza, no la CA



Flujo de una aplicación Fabric con canal



Canal (Channel)

Qué es

- Subred privada dentro de la red

Qué define

- Participantes
- Ledger independiente
- MSPs válidos

Qué permite

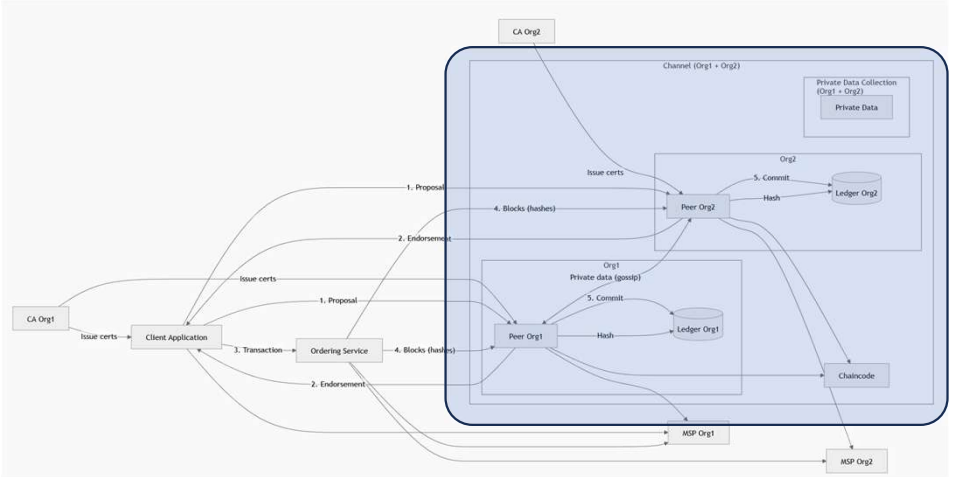
- Aislamiento total de datos

Importante

- Crear un canal requiere gobernanza
- No es dinámico “al vuelo”

Idea clave

Un canal = una red dentro de la red



PDC (Private Data Collection)

Qué es

- Configuración de privacidad dentro de un canal

Qué define

- Qué datos son privados
- Qué organizaciones los ven

Cómo funciona

- Datos → solo peers autorizados
- Hash → visible para todos

Tecnología

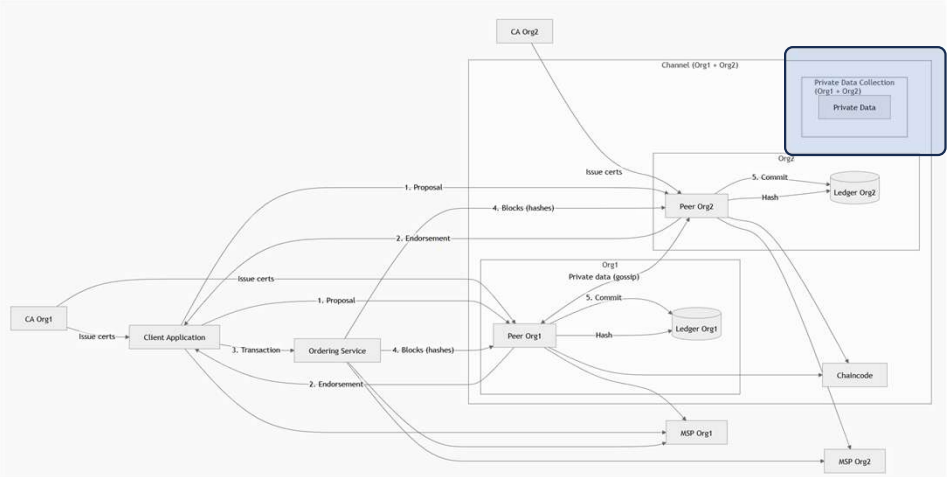
- Usa gossip para distribución

Importante

- ❌ No cifra por sí mismo
- ✓ Controla visibilidad

Idea clave

El PDC define quién ve los datos, no cómo viajan



Canal vs PDC (la diferencia clave)

Canal (Channel)

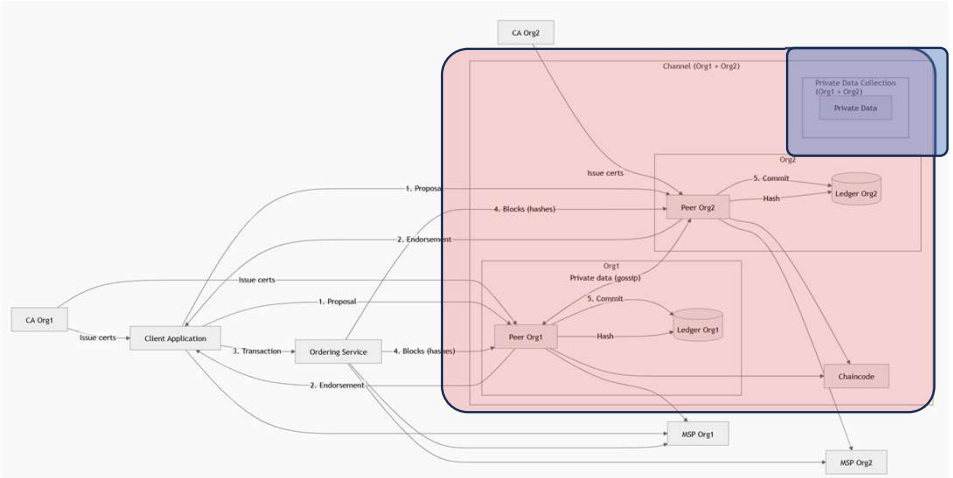
- Define **quién participa**
- Cada canal tiene:
  - su propio ledger
  - su propia configuración
- Aísla completamente la información

👉 Es como crear una red independiente

PDC (Private Data Collection)

- Define **quién ve qué datos dentro del canal**
- Solo algunos peers reciben los datos
- El resto solo ve un **hash**

👉 Privacidad sin crear nuevos canales



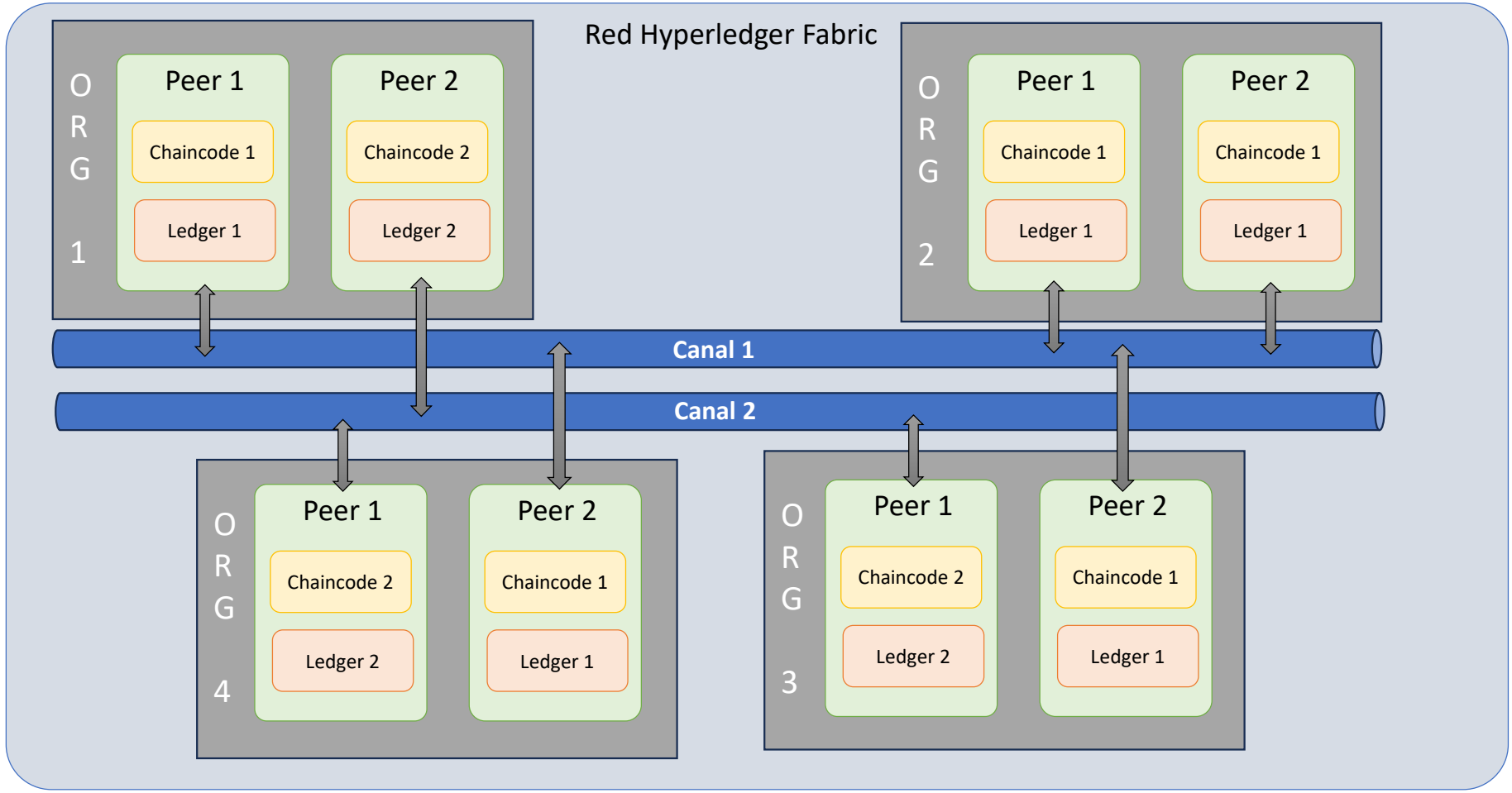
Sin PDC, todos ven todo dentro del canal; con PDC, solo ven lo que les corresponde.

Analogía

El Canal en un grupo de WhatsApp

El PDC permite mensajes privados dentro del grupo

Esquema lógico de varias organizaciones



Blockchain

Criptografía: certificados e identidades

CA vs MSP: identidad vs confianza

- En Hyperledger Fabric, la identidad se basa en estándares clásicos de PKI (Public Key Infrastructure) utilizando certificados X.509.

- La Certificate Authority (CA) es el componente que emite identidades. Cada organización tiene su propia CA, que genera certificados digitales para usuarios, peers o aplicaciones. Estos certificados permiten firmar transacciones y demostrar criptográficamente quién eres dentro de la red.

- A primera vista, podría parecer que basta con conocer el certificado raíz (root CA) para confiar en cualquier identidad emitida por esa organización. Y es cierto que con el root CA puedes verificar que un certificado es auténtico. Pero Fabric necesita algo más que autenticidad: necesita control y gobernanza.

- Ahí entra el MSP (Membership Service Provider). El MSP es una configuración distribuida en la red que define qué autoridades certificadoras son de confianza y bajo qué condiciones. No solo incluye el root CA, sino también certificados intermedios, listas de revocación y reglas de validación.

Idea clave

La CA emite identidades; el MSP define si esas identidades son válidas y confiables en la red.

Validación de identidad y por qué el MSP es necesario

Cuando un cliente envía una transacción, la firma con su certificado (emitido por la CA de su organización). Al recibirla, un peer no contacta con esa organización ni con su CA. Toda la validación se realiza localmente, usando el MSP configurado en el canal.

El peer toma el certificado y el identificador de organización (MSP ID), busca el MSP correspondiente y verifica tres cosas: que el certificado está firmado por una CA confiable, que no ha sido revocado y que la firma es correcta.

Aquí está el matiz importante: aunque el root CA permite verificar la autenticidad del certificado, no es suficiente para operar una red real.

El MSP añade capacidades esenciales:

- Permite revocar identidades comprometidas
- Define roles y permisos (admin, peer, cliente)
- Soporta jerarquías de certificación (intermediate CAs)
- Permite actualizar la confianza mediante gobernanza del canal

Sin el MSP, cualquier certificado emitido por una CA seguiría siendo válido incluso si ya no debería serlo, y no habría forma de aplicar reglas de control o acceso.

Idea clave

El certificado demuestra quién eres; el MSP valida que esa identidad es auténtica y sigue siendo válida.

¿Cuándo se aplican las Policies en Fabric?

Endorsement Policy (Chaincode)

- Durante la ejecución de la transacción
- Define qué organizaciones deben firmar
- Se evalúa antes de enviar al orderer

"¿Tengo suficientes firmas para esta operación?"

Validación en commit (Peers)

- Cuando llega el bloque desde el orderer
- Cada peer verifica que la transacción cumple la endorsement policy

Si no cumple → transacción inválida

Policies del canal (configuración)

Al modificar la red:

- añadir organizaciones
- cambiar MSPs
- actualizar configuración

Requieren firmas según reglas (ej: mayoría)

ACLs (Access Control Lists)

Controlan acceso a operaciones:

- invocar chaincode
- consultar datos
- instalar contratos

Basadas en identidad y rol (MSP)

Las policies en Fabric se aplican en distintos puntos del sistema para validar acciones mediante firmas.